Modul 9 Static Class dan Collection

Tujuan Praktikum
1. Praktikan dapat menjelaskan pengertian inner class dan menjelaskan penggunaannya. 2. Praktikan dapat menjelaskan pengertian static variable dan static method dan menjelaskan penggunaannya. 3. Praktikan dapat menjelaskan pengertian collection dan menjelaskan jenis-jenis collection serta perbedaan penggunaannya. 4. Praktikan dapat menjelaskan pengertian generics dan menjelaskan penggunaan generics..

9.1 Static

9.1.1 Static Variabel

Static variabel yaitu pendefinisian variabel milik kelas yang tidak memperlukan *instant object* untuk mengacunya. Variabel ini dapat digunakan bersama oleh semua *instant object*. Static variabel juga biasa disebut variabel *class*, serta *local* variabel tidak dapat didefinisikan sebagai static.

Contoh penggunaan static variabel sebagai berikut:

```
public class Share{
	private int privateInt;
	private static int staticInt;
	public Share(int pr, int si){
		privateInt = pr;
		staticInt = si;
	}
	public String toString(){
		return privateInt +””+ staticInt;
	}
}

public class Driver{
	public static void main(String args[]){
		Share s1 = new Share(4,4);
		System.out.println(s1.toString());
		Share s2 = new Share(8,2);
		System.out.println(s1.toString());
		System.out.println(s2.toString());
		Share s3 = new Share(6,22);
		System.out.println(s1.toString());
		System.out.println(s2.toString());
		System.out.println(s3.toString());
	}
}
```

Output :

```
> 4 4

> 4 2
> 8 2

> 4 22
> 8 22
> 6 22
```

9.1.2 Static Method

Static *method* merupakan pendefinisian *method* (milik) kelas, dengan demikian tidak memerlukan *instant object* untuk menjalankannya. *Method* ini tidak dapat menjalankan *method* yang bukan static serta tidak dapat mengacu variabel yang bukan static.

Contoh penggunaan static *method* adalah sebagai berikut :

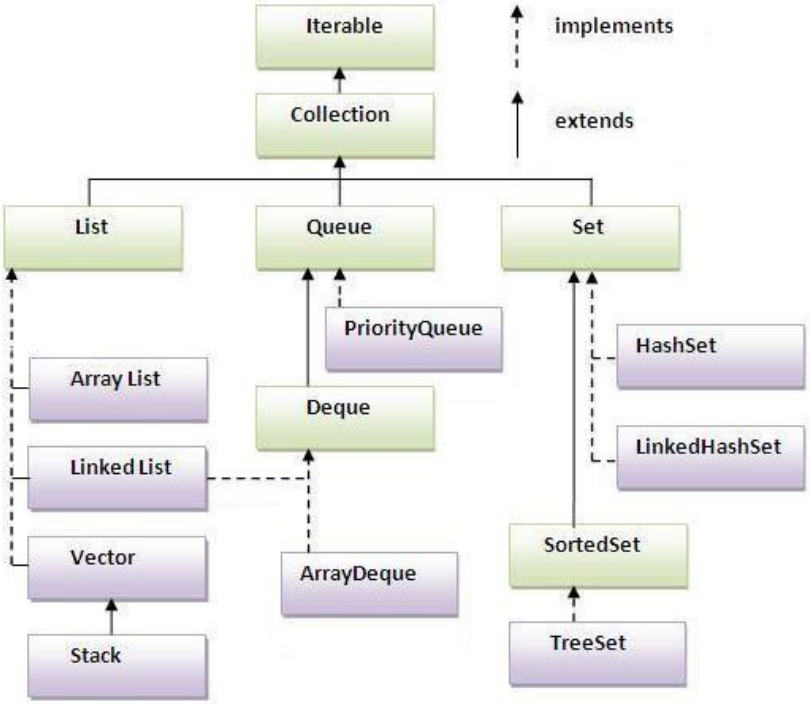
```
public class CounterMachine{
       public static void count(){
            counter++;}
}

public class Driver{
      public static void main(String args[]){
            for(int i = 0; i< 10; i++){
                  if( i % 2 == 0){
                        CounterMachine.count();
                  }
            }
            System.out.println(CounterMachine.counter);
      }
}
```

9.2 Collection

Collection adalah objek khusus yang digunakan untuk menampung banyak objek yang bertipe apapun. *Collection* dapat menampung banyak objek bertipe *Object*. Seperti diketahui, semua kelas di Java merupakan turunan dari kelas *Object*, maka dengan prinsip polimorfisme, *collection* dapat menampung objek bertipe apapun.

Collection didefinisikan sebagai *interface*. Kita dapat menggunakan kelas turunan dari *Collection*. Contoh hierarki yang ada pada *Collection* dapat dilihat pada Gambar 9-1.



Gambar 9-1 Hierarki dari Collection.

Dengan menggunakan *generics*, Kita dapat membatasi tipe yang dapat ditampung oleh *Collection*. Contohnya sebagai berikut.

```
Collection<String> str = new HashSet<String>();
List<Integer> arr_i = new List ();
ArrayList<Employee> emp = new ArrayList ();
```

Beberapa *method* yang ada pada *Collection* antara lain:

Tabel 9- 1 Beberapa Method Collection

Nama Method	Deskripsi
boolean add(Object item)	Menambahkan item ke dalam *collection*
boolean remove(Object item)	Menghapus item dari dalam *collection*
boolean addAll(Collection c)	Menambahkan semua item dari c ke dalam *collection*
boolean removeAll(Collection c)	Menghapus semua item yang ada pada c dari dalam *collection*
boolean retainAll(Collection c)	Menghapus semua item yang tidak ada pada c dari dalam *collection*
boolean contains(Object item)	Mengecek apakah item ada di dalam *collection*

Terdapat beberapa cara yang dapat digunakan untuk mengakses semua elemen dari *Collection*.

```
import java.util.*;

public class JavaApplication1 {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add(200);
        list.add("Hello");
```

```
        list.add(235);
        list.add("Donny");

        // normal loop
        System.out.println("Normal loop");
        for (int i = 0; i < list.size(); i++) {
            Object o = list.get(i);
            System.out.println(o);
        }

        // loop using for-element
        System.out.println("for-element loop");
        for (Object o : list) {
            System.out.println(o);
        }

        // Loop using iterator
        System.out.println("loop using iterator");
        Iterator itr = list.iterator();
        while (itr.hasNext()) {
            Object o = itr.next();
            System.out.println(o);
        }

        // loop using lambda Expression
        System.out.println("loop using lambda expression");
        list.forEach(o -> System.out.println(o));

        // loop using reference
        System.out.println("loop using reference");
        list.forEach(System.out::println);
    }
```

9.2.1 Melakukan Sorting pada Collection

Untuk melakukan *sorting* pada *Collection*, Kita dapat menggunakan dua acara:

a. Menggunakan *interface Comparable*

   Untuk melakukan *sorting*, dapat menggunakan *method* `Collections.sort(list)` atau `list.sort()`. Jika *list* mengandung elemen bertipe *String*, maka *list* akan diurutkan berdasarkan urutan abjad. Jika *list* mengandung elemen bertipe *Date*, maka *list* akan diurutkan berdasarkan urutan tanggal. Untuk membangun pengurutan sendiri pada *list* bertipe kelas tertentu, maka kelas tersebut harus mengimplementasikan *interface Comparable* dan meng-*override method* `compareTo(Object obj)`. *Method* `compareTo(Object obj)` akan mengembalikan:

   - Nilai < 0, jika objek yang memanggil diurutkan sebelum objek obj
   - Nilai = 0, jika objek yang memanggil sama dengan objek obj
   - Nilai > 0, jika objek yang memanggil diurutkan setelah objek obj

Jika parameter pada `compareTo()` adalah *String* atau *Date*, maka pada `compareTo()` panggil:

```
return this.getString().compareTo(obj.getString())
```

Jika parameter pada `compareTo()` adalah angka, maka pada `compareTo()` melakukan pengurangan, seperti:

```
return this.getNumber() - obj.getNumber()
```

Contoh penggunaan *interface* Comparable sebagai berikut.

```
import java.util.*;
public class Employee implements Comparable<Employee> {
    private String name;
    private int salary;
    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() {
        return name;
    }

    public int getSalary() {
        return salary;
    }

    @Override
    public String toString() {
        return "name=" + name + ", salary=" + salary;
    }

    @Override
    public int compareTo(Employee e) {
        return name.compareTo(e.name);
    }
    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList();
        listEmp.add(new Employee("bobby", 3000));
        listEmp.add(new Employee("erick", 1600));
        listEmp.add(new Employee("rey", 2500));
        listEmp.add(new Employee("anna", 3500));
        Collections.sort(listEmp);
        listEmp.forEach(System.out::println);
    }
}
```

Output yang muncul sebagai berikut:

```
name=anna, salary=3500.0
name=bobby, salary=3000.0
name=erick, salary=1600.0
name=rey, salary=2500.0
```

b. Menggunakan interface Comparator<T>

Interface comparable memungkinkan untuk melakukan *sorting* pada satu *property* saja. Untuk melakukan sorting pada *multiple property*, gunakan *interface* Comparator. Caranya adalah dengan membuat kelas yang mengimplementasikan *interface* `Comparator<T>` dan meng-*override Method* compare(T obj1, T obj2). Contohnya Kita dapat membangun kelas `comparator<>` untuk kelas `Employee()` sebagai berikut:

```
import java.util.*;
public class SalaryComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return e1.getSalary() - e2.getSalary();
    }
}
```

Penggunaan kelas Comparator pada *method* main sebagai berikut.

```
public static void main(String[] args) {
    List<Employee> listEmp = new ArrayList();
    listEmp.add(new Employee("bobby", 3000));
    listEmp.add(new Employee("erick", 1600));
    listEmp.add(new Employee("rey", 2500));
    listEmp.add(new Employee("anna", 3500));
    Collections.sort(listEmp);
    System.out.println("Sorted by name");
    listEmp.forEach(System.out::println);
    Collections.sort(listEmp, new SalaryComparator());
    System.out.println("Sorted by salary");
    listEmp.forEach(System.out::println);
}
```

Output yang dihasilkan sebagai berikut.

```
Sorted by name
name=anna, salary=3500
name=bobby, salary=3000
name=erick, salary=1600
name=rey, salary=2500

Sorted by salary
name=erick, salary=1600
name=rey, salary=2500
name=bobby, salary=3000
```

```
name=anna, salary=3500
```

9.2.2 Melakukan Filtering pada Collection

Untuk melakukan *filtering* pada *Collection*, Kita dapat menggunakan dua cara. Misalkan Kita ingin melakukan *filtering* pada *list Employee* berdasarkan kriteria tertentu. Misalkan Kita ingin mencari daftar *Employee* yang memiliki gaji minimal 3000. Maka, Kita dapat menggunakan dua cara.

a. Menggunakan cara konvensional seperti berikut.

```
System.out.println("Employee with salary minimal 3000");
for(Employee e: listEmp) {
      if(e.getSalary() >= 3000) {
            System.out.println(e);
       }
}
```

b. Menggunakan ekspresi lambda seperti berikut.

```
System.out.println("Employee with salary minimal 3000");
List<Employee> temp = listEmp.Stream().
    filter(e -> e.getSalary()>= 3000).collect(Collectors.toList());
temp.forEach(System.out::println);
```

School of Computing
Telkom University

Atau Kita ingin mencari **Employee** dengan nama tertentu. Maka Kita dapat menggunakan dua cara:

a. Menggunakan cara konvensional seperti berikut.

```
System.out.println("Employee erick");
for(Employee e: listEmp) {
      if(e.getName().equals("erick")) {
            System.out.println(e);
      }
}
```

b. Menggunakan ekspresi lambda seperti berikut.

```
System.out.println("Employee erick");
System.out.println(listEmp.Stream().
     filter(e -> e.getName().equals("erick")).
         findFirst().orElse(null));
```